

ClockAlarm - Консольный будильник на Python

Цель проекта Создать консольное приложение-будильник с упором на **оптимизацию использования системных ресурсов**.

Проект демонстрирует практическое применение стандартных библиотек Python (`datetime`, `time`) и сторонней библиотеки `pygame` для работы со звуком. Ключевой задачей было реализовать логику, при которой программа не потребляет ресурсы процессора в режиме ожидания.

Ключевые технические особенности:

- **Эффективное ожидание:** вместо постоянного опроса системного времени в цикле, что создавало бы нагрузку на CPU, приложение использует функцию `time.sleep()`. Программа переходит в спящий режим и «просыпается» ровно за 1 секунду до установленного времени будильника.
 - **Гибкая настройка сигнала:** реализована возможность выбора типа побудки — беззвучный режим (с выводом текстового сообщения) или одна из предустановленных мелодий.
 - **Интерактивное управление:** в момент звонка пользователь может отложить сигнал на 5 минут (до 5 раз подряд) или полностью его остановить.
 - **Надёжность:** программа включает проверку вводимых пользователем данных на корректность (дата, время) и обработку исключений для предотвращения сбоев.
-

Достигнутые результаты

В рамках проекта был разработан полнофункциональный будильник, который эффективно решает поставленные задачи:

1. **Реализован механизм «спящего режима»**, позволяющий приложению не использовать процессорное время во время ожидания. Это достигается расчётом интервала сна до секунды перед срабатыванием будильника.
 2. **Обеспечена поддержка нескольких будильников.** Пользователь может последовательно установить несколько событий.
 3. **Создана система управления сигналом**, позволяющая откладывать или останавливать будильник после его активации.
 4. **Добавлена валидация ввода** для защиты от некорректно заданных дат и времени.
-

Установка мелодии и работа будильника

```
(.venv) PS C:\Users\Evgeny\Mini_S\PythonProjects\ClockAlarm> py -m clockalarm
pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)
Hello from the pygame community. https://www.pygame.org/contribute.html
ВАРИАНТЫ ПОБУДКИ:
0 - Беззвучная

1 - Louis Armstrong - Go Down Moses

2 - Matia Bazar - Vacance Romane

3 - Fallout New Vegas Radio - Jazz Blues

№ варианта: 2
Вы выбрали Matia Bazar - Vacance Romane

Установите время будильника 2026-MM-DD hh:mm
месяц MM: 06
число DD: 15
час(24) hh: 20
минут mm: 19
Будильник будет установлен!

Добавить будильник?...введите 'yes'
Продолжить - нажмите ENTER
Ваш выбор:

alarm_time: 2026-06-15 20:19
Мелодия: Matia Bazar - Vacance Romane
Чтобы остановить этот будильник, введите 'stop'.
Чтобы отложить на 5 минут нажмите ENTER.

Этот будильник отложен в 1й раз

Чтобы остановить этот будильник, введите 'stop'.
Чтобы отложить на 5 минут нажмите ENTER.

Этот будильник отложен в 2й раз

Чтобы остановить этот будильник, введите 'stop'.
Чтобы отложить на 5 минут нажмите ENTER.
stop
Этот будильник остановлен
Чтобы остановить все будильники, введите 'stop'.
Чтобы продолжить, нажмите ENTER.

(.venv) PS C:\Users\Evgeny\Mini_S\PythonProjects\ClockAlarm> |
```

Установка сообщения в беззвучном режиме

```
(.venv) PS C:\Users\Evgeny\Mini_S\PythonProjects\ClockAlarm> py -m clockalarm
pygame 2.6.1 (SDL 2.28.4, Python 3.13.7)
Hello from the pygame community. https://www.pygame.org/contribute.html
ВАРИАНТЫ ПОБУДКИ:
0 - Беззвучная

1 - Louis Armstrong - Go Down Moses

2 - Matia Bazar - Vacance Romane

3 - Fallout New Vegas Radio - Jazz Blues

№ варианта: 0
Вы выбрали Беззвучная

Задайте сообщение будильника или нажмите ENTER
Сообщение:
Ваше сообщение (по умолчанию): Время вставать!
Установите время будильника 2026-MM-DD hh:mm
месяц MM: |
```

Веб-приложение «Личный дневник» на Flask

Цель проекта Создать полноценное веб-приложение с использованием фреймворка Flask, реализующее функционал личного дневника. Ключевой задачей было освоить архитектуру *MVC* (Модель-Представление-Контроллер) в контексте Flask, научиться работать с базами данных через SQLAlchemy и реализовать систему аутентификации пользователей с управлением сессиями.

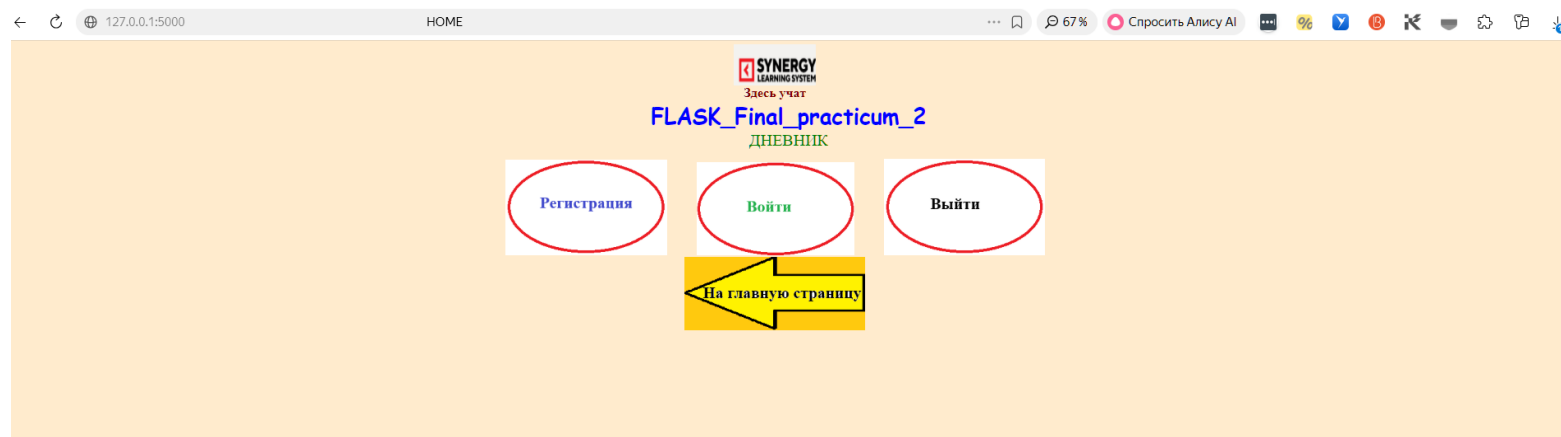
Ключевые технические особенности:

- **Бэкенд:** Разработка на Python с использованием микрофреймворка Flask.
- **База данных:** Использование SQLite для хранения данных. Управление схемой БД и миграциями осуществлялось с помощью Flask-Migrate.
- **Аутентификация:** Реализована система регистрации и входа пользователей с созданием сессий (токен действителен 30 минут). Доступ ко всем функциям приложения защищён и требует авторизации.
- **CRUD-операции:** Полный цикл операций над записями дневника: создание (*Create*), чтение (*Read*), редактирование (*Update*) и удаление (*Delete*).
- **Фронтенд:** Построение пользовательского интерфейса с помощью шаблонизатора Jinja2 и стилизация страниц с применением CSS.

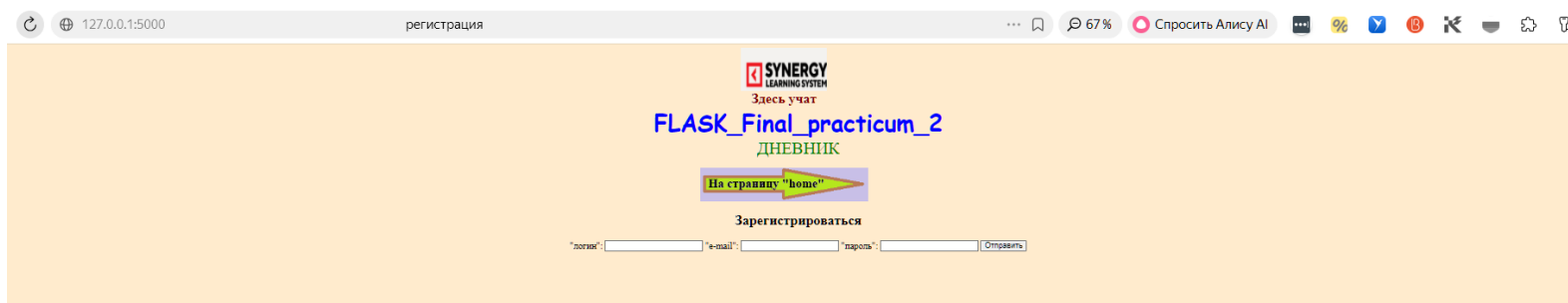
Достигнутые результаты В рамках проекта был разработан функциональный прототип веб-сервиса, который позволяет пользователям:

1. **Зарегистрироваться и войти в личный кабинет**, используя логин, email и пароль.
2. **Управлять личными записями:** создавать новые заметки, просматривать список всех записей, а также редактировать или удалять существующие по их уникальному идентификатору.
3. **Работать в безопасной сессии.** Все действия пользователя защищены, а время жизни сессии ограничено для обеспечения базовой безопасности.
4. **Использовать адаптивный интерфейс.** Приложение корректно отображается на различных устройствах благодаря использованию мета-тега `viewport` и продуманной верстке.

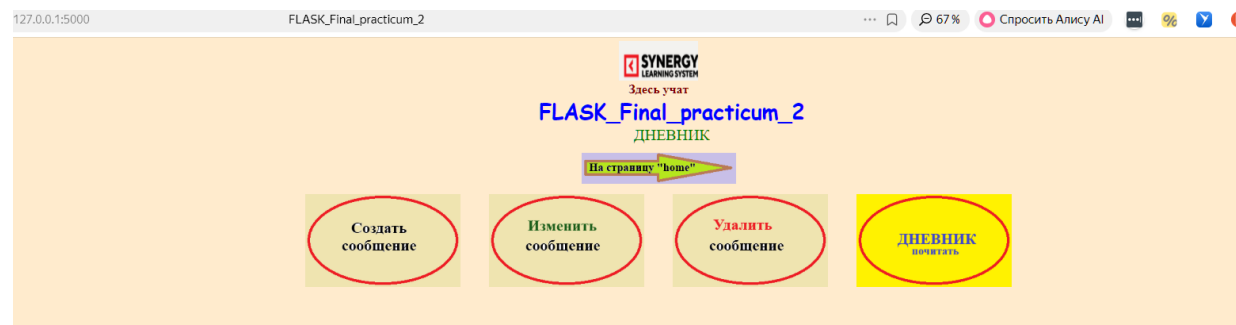
1. Страница вход/выход/регистрация (/home)



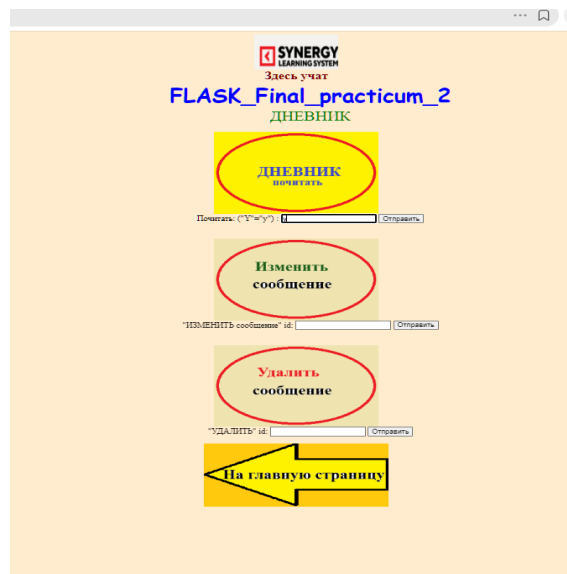
2. Страница регистрации (/register).



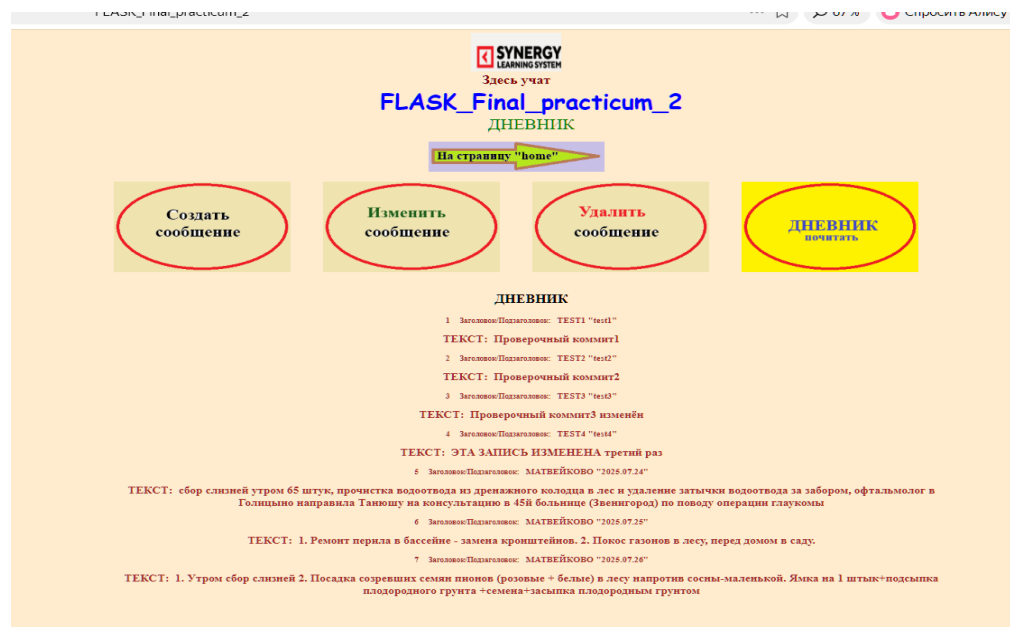
3. Основной экран дневника (/) - управление записями



4. Страница выбора действий с записями (/card)



Почитать дневник



ИЗМЕНИТЬ сообщение (изменение 4й записи).

FLASK_Final_practicum_2

SYNERGY
LEARNING SYSTEM
Здесь учат

FLASK_Final_practicum_2
ДНЕВНИК

На главную страницу

Создать сообщение

Изменить сообщение

Удалить сообщение

ДНЕВНИК
прочитать

Запись в базе:
4 Заголовок/Подзаголовок: TEST4 "test4"

ТЕКСТ: ЭТА ЗАПИСЬ ИЗМЕНЕНА третий раз

Изменить на (введите id, незаполненные поля не будут изменены):

"ПОДТВЕРДИТЕ ИЗМЕНИТЬ" id 4: "Заголовок": "Подзаголовок": "Текст": ЭТА ЗАПИСЬ ИЗМЕНЕНА

5. Создать сообщение (/form_create)

form_create

SYNERGY
LEARNING SYSTEM
Здесь учат

FLASK_Final_practicum_2
ДНЕВНИК

Заголовок

Подзаголовок Текст

ТЕКСТ

На главную страницу

DjangoMoneyCollect - REST API для групповых денежных сборов на Django

Цель проекта Разработать и реализовать полноценное REST API для платформы по организации групповых денежных сборов (краудфандинга). Проект нацелен на создание серверной части, которая предоставляет структурированный доступ к данным о сборах, платежах и пользователях, а также реализует бизнес-логику взаимодействия между ними.

Ключевые технические особенности:

- **Архитектура:** Проект построен на базе Django REST Framework (DRF), что позволило быстро создать стандартизированное и документированное API.
- **Асинхронность:** Для выполнения фоновых задач (отправка email-уведомлений) реализована интеграция с Celery и Redis, что обеспечивает высокую отзывчивость основного API. Так как API создавалось в среде Windows, то для совместимости Windows с Celery и эмуляции многопоточности был выбран асинхронный пул `gevent` с сопутствующим «monkey-патчиванием» штатных библиотек.
- **Масштабируемость и развертывание:** Проект полностью готов к контейнеризации с помощью Docker и Docker-Compose, что упрощает его развертывание и масштабирование в различных средах.
- **Безопасность:** Реализована система авторизации и разграничения прав доступа к эндпоинтам API. Доступ к административным функциям ограничен.

Достигнутые результаты В рамках проекта был разработан функциональный бэкенд, предоставляющий следующие возможности:

1. **Управление сборами:** Создание, чтение, обновление и удаление (CRUD) целей сборов. Доступен отдельный эндпоинт для получения списка активных сборов.
2. **Обработка платежей:** Реализован функционал для совершения пожертвований в пользу конкретного сбора и просмотра истории платежей.
3. **Управление пользователями:** Создание и просмотр пользователей системы.
4. **Гибкий поиск и фильтрация:** Клиенты API могут фильтровать сборы по категории (например, благотворительность), выполнять полнотекстовый поиск по названию и описанию, а также сортировать результаты.
5. **Автоматизация:** Настроена автоматическая отправка email-уведомлений участникам сборов при создании или обновлении событий.
6. **Тестовые данные:** Реализован инструмент `Management command` генерации моковых данных в режиме отладки (`generate_mock_data.py`) для быстрого наполнения базы данных большим количеством записей (пользователи, сборы, платежи...) с применением пакетных SQL-запросов.

Демонстрация работы API.

1. Список доступных эндпоинтов API

← ↻ 127.0.0.1:8000 Api Root – Django REST framework ⋮ 110 % Перевести Спросить Алису AI

Django REST framework Log in

Schema / Api Root

Api Root

The default basic root view for DefaultRouter

GET /api/

HTTP 200 OK

Allow: GET, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "users": "http://127.0.0.1:8000/api/users/",
  "collects": "http://127.0.0.1:8000/api/collects/",
  "payments": "http://127.0.0.1:8000/api/payments/"
}
```

2. Главная страница SwaggerUI (/swagger/)

The screenshot displays the SwaggerUI interface for the "Money Collect API". The browser's address bar shows the URL "127.0.0.1:8000". The SwaggerUI header includes the API title "Money Collect API" and a version number "1.0.0". Below the header, there is a "Filter by tag" input field. The main content area is divided into three sections: "collects", "payments", and "users". Each section lists the available endpoints with their corresponding HTTP methods (GET, POST, PUT, PATCH, DELETE) and the endpoint name. The "collects" section includes endpoints for listing, creating, updating, deleting, and donating. The "payments" section includes endpoints for listing, creating, updating, deleting, and donating. The "users" section includes endpoints for listing, creating, updating, deleting, and donating. The interface is clean and modern, with a light gray background and blue accents.

Money Collect API

SwaggerUI

127.0.0.1:8000

50% Сбросить

Money Collect API

1.0.0

API для сбора денег

Contact the developer

BSD License

Filter by tag

collects

GET /collects/ collects_list

POST /collects/ collects_create

GET /collects/active/ collects_active

GET /collects/{id}/ collects_read

PUT /collects/{id}/ collects_update

PATCH /collects/{id}/ collects_partial_update

DELETE /collects/{id}/ collects_delete

POST /collects/{id}/donate/ collects_donate

GET /collects/{id}/payments/ collects_payments

payments

GET /payments/ payments_list

POST /payments/ payments_create

GET /payments/{id}/ payments_read

PUT /payments/{id}/ payments_update

PATCH /payments/{id}/ payments_partial_update

DELETE /payments/{id}/ payments_delete

users

GET /users/ users_list

POST /users/ users_create

GET /users/me/ users_me_read

POST /users/me/ users_me_create

PUT /users/me/ users_me_update

DELETE /users/me/ users_me_delete

GET /users/{id}/ users_read

16°C В осн. солнечно 14:48 16.06.2026

3. Документация эндпоинта создания взноса в сбор.

The screenshot displays the documentation for the Money Collect API, specifically the **POST /payments/** endpoint. The interface is divided into several sections:

- Parameters:** A section for defining request parameters. It includes a "Name" column and a "Description" column. A "data" parameter is defined as a required object (body) with an example value and a model. The example value is a JSON object:

```
{  "collect": 0,  "user": 1,  "amount": "string",  "comment": "string",  "is_anonymous": true}
```

. The parameter content type is set to `application/json`.
- Responses:** A section for defining response parameters. It includes a "Code" column and a "Description" column. A response with status code **201** is defined with an example value and a model. The response content type is set to `application/json`.
- Code:** A section for defining the response body. It includes a "Code" column and a "Description" column. A response with status code **201** is defined with an example value and a model. The example value is a JSON object:

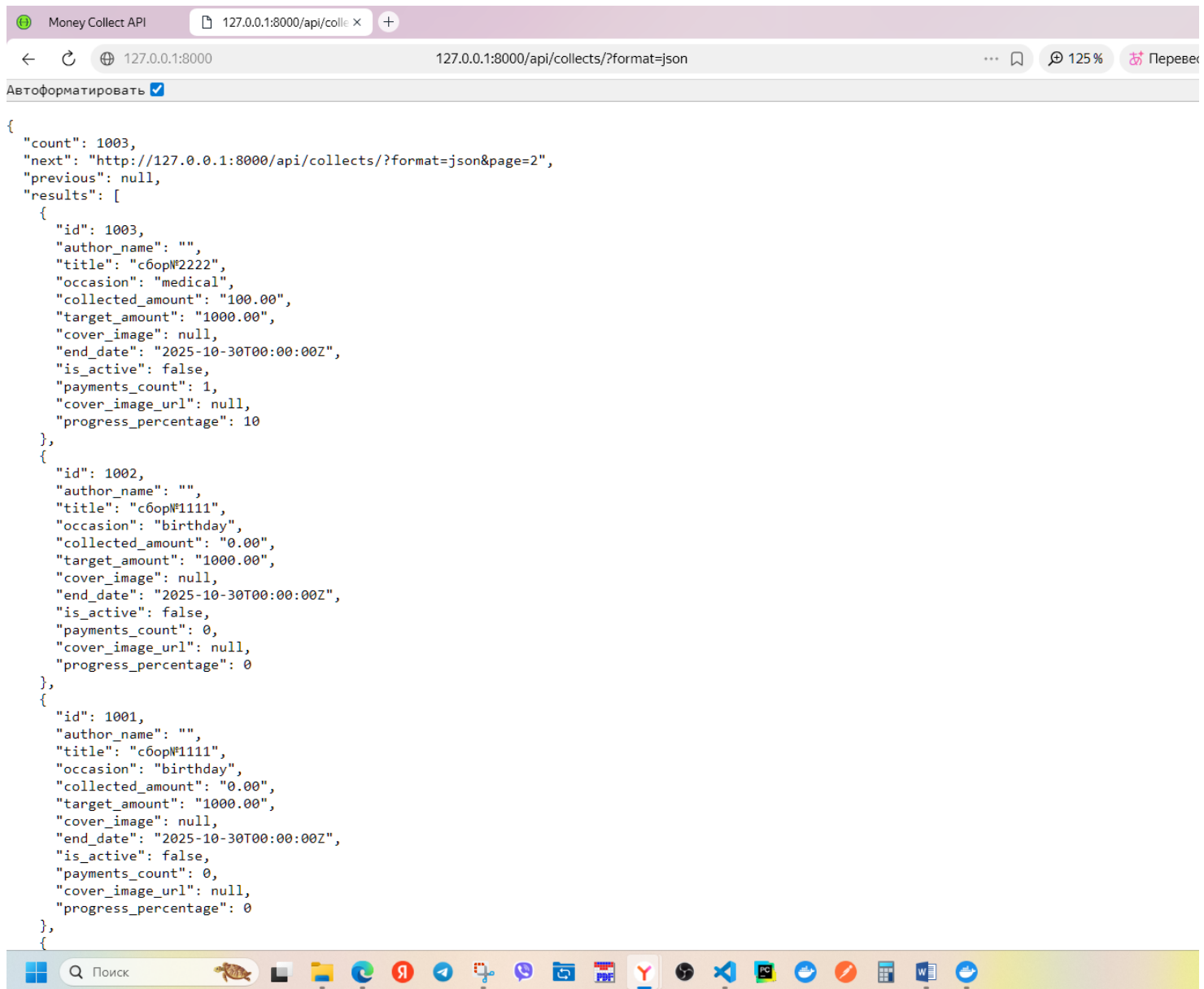
```
{  "id": 1,  "collect": 0,  "collect_title": "string",  "user": 1,  "user_name": "string",  "user_username": "string",  "amount": "string(5decimal)",  "comment": "string",  "created_at": "string(datetime)",  "is_anonymous": true}
```

. The response content type is set to `application/json`.

At the bottom of the interface, there are tabs for other endpoints: **GET /payments/{id}/** (payments_read) and **PUT /payments/{id}/** (payments_update).

The Windows taskbar at the bottom shows the search bar, task view, and various application icons. The system tray on the right shows the temperature (16°C), weather (В осн. солнечно), and the date and time (15:59, 16.06.2026).

4. Пример ответа API (список сборов).



DatingApp - API для платформы знакомств (DatingApp) на Django

Цель проекта Разработать REST API для веб-платформы знакомств, реализующий полный цикл взаимодействия между пользователями: от регистрации и поиска до общения в чатах. Ключевой задачей было создание сложной системы взаимодействий (лайки, дизлайки, мэтчи, приглашения) и обеспечение безопасной авторизации через различные каналы (email/пароль, социальные сети, JWT).

Ключевые технические особенности:

- **Архитектура:** Проект построен на Django с использованием Django REST Framework (DRF) для создания API. В качестве базы данных используется PostgreSQL.
- **Аутентификация:** Реализована гибкая система входа с поддержкой JWT-токенов (SimpleJWT) и OAuth2-авторизации через социальные сети (Google, Yandex, Mail.ru) с помощью пакета django-allauth.
- **Взаимодействия:** Сложная логика взаимодействий между пользователями: система лайков и дизлайков, формирование взаимных симпатий (мэтчей), история просмотров, приглашения на свидания и защищенный обмен контактами.
- **Реальное время:** Для реализации чатов между пользователями применена технология WebSocket через Django Channels.
- **Докеризация:** Проект полностью готов к развертыванию в изолированной среде с помощью Docker и Docker Compose. Для удобства работы с базой данных в режиме отладки в стек включён pgAdmin 4, доступный по адресу <http://127.0.0.1:5050> после запуска контейнеров.

Достигнутые результаты В рамках проекта был разработан функциональный бэкенд, предоставляющий следующие возможности:

1. **Регистрация и авторизация:** Пользователи могут создавать аккаунты через email/пароль или входить с помощью учетных записей социальных сетей. При любом способе входа выдается JWT-токен для доступа к защищенным эндпоинтам.
2. **Система поиска и фильтрации:** Реализован механизм подбора случайных профилей с возможностью фильтрации по полу, возрасту, городу и статусу отношений.
3. **Механизм взаимодействий:** Пользователи могут ставить лайки и дизлайки другим профилям. При взаимном лайке формируется «мэтч», который открывает доступ к личному чату.
4. **Лента событий:** Доступна история просмотренных профилей, полученных лайков и отправленных дизлайков.
5. **Чаты (WebSocket):** Пользователи могут общаться в личных чатах со своими «мэтчами» в режиме реального времени с сохранением истории сообщений в базе данных.
6. **Приглашения и обмен контактами:** Реализована система отправки приглашений на свидание или предложения об обмене контактами, которая требует взаимного подтверждения для безопасности.
7. **Генерация тестовых данных:** Реализован инструмент Management command генерации моковых данных в режиме отладки (`generate_mock_data.py`) для быстрого наполнения базы данных большим количеством пользователей и их взаимодействий с применением пакетных SQL-запросов.

I. Демонстрация работы API.

1. Главная страница (/)

Общедоступные эндпойнты (пользователь не зарегистрирован)

Dating Platform API

API для платформы знакомств | Версия 1.0.0

Не авторизован

Аутентификация

Для доступа к API потребуется JWT (токен выдается автоматически при авторизации через /api-auth/login/ или можно скопировать через /api/token/ для зарегистрированных пользователей).

Быстрый вход с тестовыми учетными записями и просмотр документации (dev-режим)

Быстрый вход с Test User

Быстрый вход с Admin

Swagger Docs

Тестовые учетные данные:

Test User: test@test.ru / password

Admin: admin@admin.ru / password

Доступные эндпоинты

Для ВСЕХ АВТОРИЗАЦИЯ/РЕГИСТРАЦИЯ/ДОКУМЕНТАЦИЯ

комбо: WEB вход по логин-паролю|WEB и JWT социализация|Django-регистрация|токен не создается|удаление аккаунта

/login/

GETPUT

Страница входа(регистрации)/удаления аккаунта

получить jwt-токен для заголовка headers: "Authorization": "Bearer \${token}"

/api/token/

POST

Получить JWT токены (для зарегистрированных пользователей)

вход по паролю WEB (Browsable API) токен-авторизация

/api-auth/login/

GET

Вход через сессию (для зарегистрированных пользователей)

WEB-авторизация|WEB-регистрация|WEB-авторизация через соцсети

/accounts/login/

GET

WEB Вход/регистрация

Django-регистрация

/register/

POST

Django-регистрация (в сессии, токен не создается)

documentation swagger

/swagger/

GET

Интерактивная документация API

documentation redoc

/redoc/

GET

Интерактивная документация API

2. Главная страница (/)

все эндпойнты (для админа)

 **Dating Platform API**

API для платформы знакомств | Версия 1.0.0

API-ключы: admin@demo.ru

Войти

Аутентификация

Для доступа к API требуется JWT (token выдается автоматически при авторизации через api/auth/login или можно сгенерировать через endpoint для зарегистрированных пользователей).

Быстрый вход в тестовые учетные данные и просмотр документации (де-режим)

[Назад к Testnet](#) [Войти в Admin](#) [Просмотр Docs](#)

Тестовые учетные данные:

Test user: test@demo.ru / password

Admin: admin@demo.ru / password

Доступные эндпоинты

для ВСЕХ АВТОРИЗАЦИОННЫХ РЕГИСТРАЦИОННЫХ ДОКУМЕНТАЦИЙ

POST /api/v2b/uid по почте получить WEB и API идентификационный документ регистрации через создание учетной записи

[POST](#)

Страница кодирования идентификационного документа

Анонимный URL-адрес для запросов к сервису "Authenticator" - "Basic Token"

[POST](#)

Получить 2017 токены (для зарегистрированных пользователей)

код по паролю WEB (Приватный API) token-аутентификация

[GET](#)

Вход через email (для зарегистрированных пользователей)

WEB-идентификация WEB-регистрация WEB-аутентификация через создание

[POST](#)

WEB-идентификация

Одноразовая идентификация

[POST](#)

Одноразовая идентификация (в основном, только не создается)

documentatn swagger

[Swagger?](#)

API

Идентификация документации API

documentatn redis

[Redis](#)

Идентификация документации API

ЗАЩИЩЕННЫЕ АЛГЕБРАИЧЕСКИЕ ЭНДПОИНТЫ

api/api/uid/uid

[GET](#)

Обновить 2017 access token

api/api/uid

[GET](#)

Вывести api access token

profile

[GET](#)

[PUT](#)

[DELETE](#)

Профиль текущего пользователя

received likes

[GET](#)

Все лайки пользователя

random profile

[GET](#)

Случайный профиль для лайков

username list

[GET](#)

Список чужих пользователей

ADMIN

admin panel

[Admin?](#)

[GET](#)

Панель администратора

user list

[GET](#)

Список всех пользователей (только для администраторов)

[illegible]

3. Вход/регистрация (/login/)

Вход в систему

Войти

Войти через Google (WEB)

Войти через Google (JWT)

Войти через Yandex(WEB)

Войти через Yandex (JWT)

Войти через Mail.ru(WEB)

Войти через Mail.ru (JWT)

[Нет аккаунта? Зарегистрироваться \(без выдачи JWT\)](#)

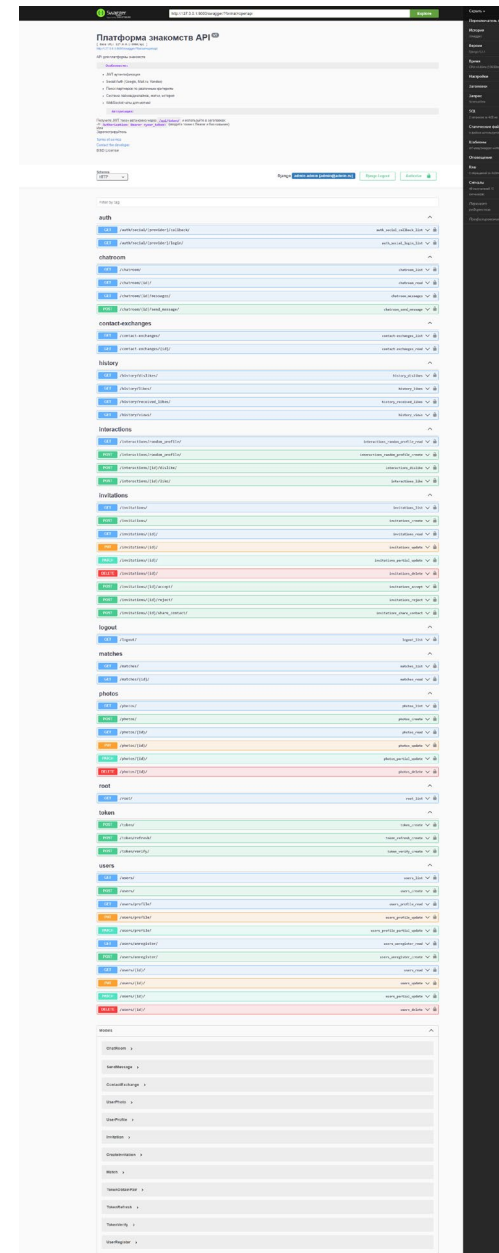
Тестовые данные:

👑 Суперпользователь: admin@admin.ru / password

👤 Тестовые пользователи: user_{id}@ example.com / password

🗑 [Удалить аккаунт](#) | [API версия](#)

4. Документация Swagger UI (/swagger/)



5. Документация эндпоинта получения случайного профиля (/swagger).

GET

/interactions/random_profile/

interactions_random_profile_read

Получение случайного профиля с фильтрами

В Browsable API: отображает профиль и формы для like/dislike

Parameters

Cancel

No parameters

Execute

Clear

Responses

Response content type

application/json

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/api/interactions/random_profile/' \
-H 'accept: application/json' \
-H 'X-CSRFToken: NCTtoeVdEXCC6c8356Bnim1B5TvGrzIjNJC3FH7xegitLzKXgaTv9pzmahP3sMxx'
```

Request URL

```
http://127.0.0.1:8000/api/interactions/random_profile/
```

Server response

Code

Details

200

Response body

```
{
  "user": {
    "id": 39,
    "email": "user_39@example.com",
    "first_name": "user_39",
    "last_name": "user_39",
    "gender": "M",
    "age": 45,
    "city": "Санкт-Петербург",
    "hobbies": "\\путешествия,фотография,кулинария\\",
    "status": "single",
    "likes_count": 0,
    "is_private": false,
    "phone": "user_39-phone: \\phone-number\\",
    "photos": [
      {
        "id": 117,
        "image": "/media/user_photos/default_avatar853459a47ac1453daf56574056b92274.jpg",
        "is_main": true,
        "created_at": "2026-06-17T11:16:18.971738Z"
      },
      {
        "id": 118,
        "image": "/media/user_photos/default_avatar1c15b945aec54ba59c709c456aa9a329.jpg",
        "is_main": false,
        "created_at": "2026-06-17T11:16:18.972402Z"
      }
    ],
    "main_photo": {
      "id": 117
    }
  }
}
```

Download

Response headers

```
allow: GET,POST,HEAD,OPTIONS
content-length: 1611
content-type: application/json
cross-origin-opener-policy: same-origin
date: Wed,17 Jun 2026 18:52:06 GMT
djdt-request-id: 5481205fab094e5a9f8d4a0e69872e73
referrer-policy: same-origin
server: WSGIServer/0.2 CPython/3.12.13
server-timing: TimerPanel_otime;dur=31.381000000010317;desc="User CPU time",TimerPanel_stime;dur=18.6909999999898;desc="System CPU time",TimerPanel_total;dur=50.072000000000116;desc="Total"
```

6. Примеры ответа API.

Эндпоинт получения профиля пользователя (всегда текущего). *Browsable API*(/api/users/profile/).

The screenshot displays a Browsable API interface for the endpoint `GET /api/users/profile/`. The breadcrumb navigation shows `Api Root / User List / Profile`. The title of the section is **Profile**. Below the title, it states "Профиль всегда текущего пользователя". The HTTP method and path are shown as `GET /api/users/profile/`. The response status is `HTTP 200 OK`, and the headers are `Allow: GET, PUT, PATCH, HEAD, OPTIONS`, `Content-Type: application/json`, and `Vary: Accept`. The response body is a JSON object representing a user profile:

```
{
  "id": 1,
  "email": "admin@admin.ru",
  "first_name": "admin",
  "last_name": "admin",
  "gender": "M",
  "age": 30,
  "city": "Москва",
  "hobbies": "",
  "status": "single",
  "likes_count": 0,
  "is_private": false,
  "phone": "",
  "photos": [],
  "main_photo": null
}
```

Below the JSON response, there are tabs for `Raw data` and `HTML form`. The `HTML form` tab is active, showing a form with the following fields:

- Имя:
- Фамилия:
- Пол: - Возраст:
- Город:
- Увлечения:
- Статус отношений: - Приватный профиль: ☐
- Телефон:

A `PUT` button is located at the bottom right of the form.

Эндпоинт полученных лайков. *Browsable API*(/history/received_likes/).

Api Root / Received likes

Received likes

История лайков профиля пользователя (кто лайкнул меня)

GET /api/history/received_likes/

HTTP 200 OK

Allow: GET, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
[
  {
    "id": 5,
    "user_from_profile": {
      "id": 31,
      "email": "user_31@example.com",
      "first_name": "user_31",
      "last_name": "user_31",
      "gender": "F",
      "age": 43,
      "city": "Казань",
      "hobbies": "\\Искусство,театр,музеи\\",
      "status": "married",
      "likes_count": 0,
      "is_private": false,
      "phone": "user_31-phone: \\*pnone-number\\\"",
      "photos": [
        {
          "id": 90,
          "image": "/media/user_photos/default_avatarfe44c409b62c4495a50c6d39fd9f071.jpg",
          "is_main": true,
          "created_at": "2026-06-17T11:16:18.952767Z"
        },
        {
          "id": 91,
          "image": "/media/user_photos/default_avatar2beb851178364841a11c0391360365da.jpg",
          "is_main": false,
          "created_at": "2026-06-17T11:16:18.953513Z"
        }
      ],
      "main_photo": {
        "id": 90,
        "image": "/media/user_photos/default_avatarfe44c409b62c4495a50c6d39fd9f071.jpg",
        "is_main": true,
        "created_at": "2026-06-17T11:16:18.952767Z"
      }
    },
    "action_type": "like",
    "created_at": "2026-06-17T11:16:19.178409Z"
  }
]
```

Extra Actions ▾

OPTIONS

GET ▾

- Dislikes
- Likes
- Received likes
- Views

II. Интерфейс pgAdmin 4, развернут при запуске docker-контейнеров проекта.

Администрирование базы данных PostgreSQL(<http://127.0.0.1:5050>).

The screenshot displays the pgAdmin 4 web interface in a browser. The address bar shows the URL `127.0.0.1:5050`. The interface includes a sidebar with an "Object Explorer" showing a database structure with various tables and schemas. The main panel shows a query executed in the `public.users_match/datingapp/postgres@Docker_datingapp` database. The query is `SELECT * FROM public.users_match ORDER BY id ASC LIMIT 100`. The results are displayed in a table with 50 rows shown. The table has columns: `id` (PK, bigint), `created_at` (timestamp with time zone), `is_active` (boolean), `user1_id` (bigint), and `user2_id` (bigint). The status bar at the bottom indicates "Total rows: 50" and "Query complete 00:00:00.250".

id	created_at	is_active	user1_id	user2_id
1	2026-06-17 11:16:19.153915+	true	2	85
2	2026-06-17 11:16:19.170651+	true	46	63
3	2026-06-17 11:16:19.18614+00	true	10	31
4	2026-06-17 11:16:19.21162+00	true	20	92
5	2026-06-17 11:16:19.226566+	true	18	66
6	2026-06-17 11:16:19.241886+	true	40	47
7	2026-06-17 11:16:19.266933+	true	96	100
8	2026-06-17 11:16:19.281918+	true	41	78
9	2026-06-17 11:16:19.306739+	true	52	97
10	2026-06-17 11:16:19.322741+	true	26	66
11	2026-06-17 11:16:19.33748+00	true	5	37
12	2026-06-17 11:16:19.352558+	true	20	53
13	2026-06-17 11:16:19.367933+	true	18	85
14	2026-06-17 11:16:19.382759+	true	6	59
15	2026-06-17 11:16:19.406958+	true	42	84
16	2026-06-17 11:16:19.419065+	true	35	94
17	2026-06-17 11:16:19.430966+	true	59	66
18	2026-06-17 11:16:19.456898+	true	58	79
19	2026-06-17 11:16:19.471201+	true	29	48
20	2026-06-17 11:16:19.483985+	true	32	83
21	2026-06-17 11:16:19.509723+	true	37	63
22	2026-06-17 11:16:19.522317+	true	59	75
23	2026-06-17 11:16:19.536015+	true	16	88
24	2026-06-17 11:16:19.562847+	true	52	89
25	2026-06-17 11:16:19.578437+	true	48	76
26	2026-06-17 11:16:19.593044+	true	17	31
27	2026-06-17 11:16:19.62105+00	true	28	39
28	2026-06-17 11:16:19.63467+00	true	14	52
29	2026-06-17 11:16:19.647828+	true	83	91
30	2026-06-17 11:16:19.674519+	true	86	94
31	2026-06-17 11:16:19.687228+	true	58	83
32	2026-06-17 11:16:19.698922+	true	15	78
33	2026-06-17 11:16:19.724154+	true	65	86
34	2026-06-17 11:16:19.735912+	true	48	85

Docker Desktop

PERSONAL

Search

Ctrl+K

?

1

Sign in

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers

Give feedback

Container CPU usage ⓘ
4.77% / 400% (4 CPUs available)

Container memory usage ⓘ
486.76MB / 7.47GB

Show charts

Search

☰

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/write	N	Actions
☐										
☐ ▾ ●	dating_app	-	-	-	14.59%	486.82MB / 22.94	6.21%	184.4MB / 285.05	22	
☐ ●	db-1	c9b7ed70c116	postgres:15	5432:5432 ↗	9.83%	71.42MB / 7.65Gi	0.91%	43.6MB / 9.29MB	1.5	
☐ ●	web-1	df26a17c61fe	dating_app	8000:8000 ↗	4.71%	148.9MB / 7.65Gi	1.9%	28.8MB / 24.8MB	1.5	
☐ ●	pgadmin4	99fb757f0995	dpage/pgad	5050:80 ↗	0.05%	266.5MB / 7.65Gi	3.4%	112MB / 251MB	19	

Engine running

RAM 2.56 GB CPU 7.58% Disk: 6.24 GB used (limit 1006.85 GB)

>_ Terminal

New version a

Showing 4 ite